

Data Engineering Approach

Pipeline Design — Enterprise Knowledge Copilot

1. Document Sources

All organizational documents are ingested from five source categories. Each source has a dedicated connector with a fixed fetch mechanism.

Source	Examples	Fetch Method	Trigger
SharePoint / OneDrive	SOPs, Policies, HR docs	Microsoft Graph API (OAuth2)	Webhook on file change
Confluence / Wiki	IT Runbooks, Project docs	Confluence REST API v2	Webhook on page publish
Local Upload	Any file type	FastAPI HTTP endpoint	On-demand (user action)
Git Repositories	Tech docs, READMEs	git pull via gitpython	Cron every 15 min
Email Attachments	Reports, invoices	IMAP via imaplib	Cron every 5 min

Every fetched document is stored immediately to MinIO (self-hosted S3-compatible object store) before any processing begins. This ensures the original is never lost.

Storage Tier	Path	Contents
Raw	s3://idp/raw/{source}/{doc_id}.{ext}	Original file — immutable, never modified
Processed	s3://idp/processed/{doc_id}/	Extracted text JSON, table JSON, image files
Metadata	PostgreSQL: documents table	doc_id, source, filename, status, timestamps

 **Dedup:** SHA-256 hash of raw bytes is checked in Redis before storing. If hash exists, skip — do not reprocess.

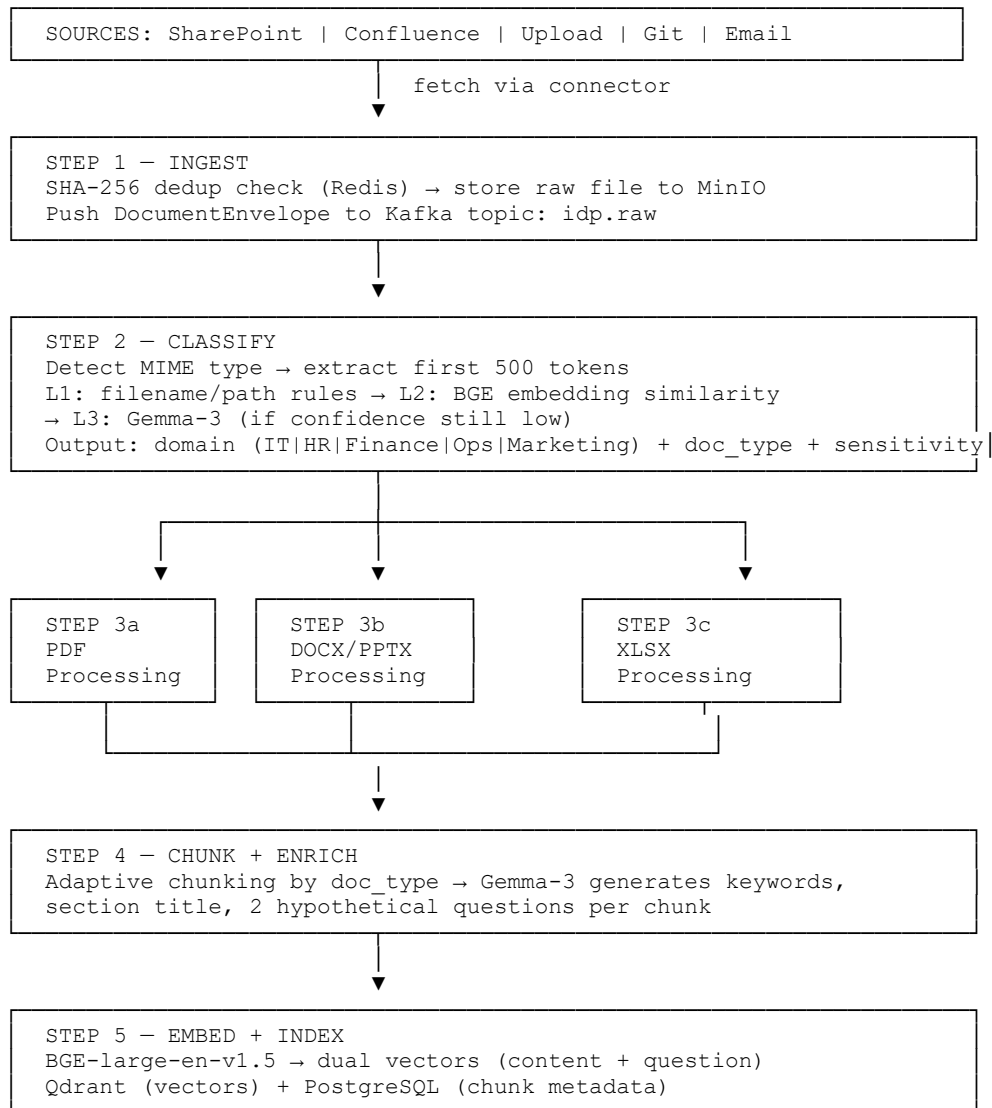
2. Where Documents Are Stored (Raw)

Every fetched document is stored immediately to MinIO (self-hosted S3-compatible object store) before any processing begins. This ensures the original is never lost.

Storage Tier	Path	Contents
Raw	s3://idp/raw/{source}/{doc_id}.{ext}	Original file — immutable, never modified
Processed	s3://idp/processed/{doc_id}/	Extracted text JSON, table JSON, image files
Metadata	PostgreSQL: documents table	doc_id, source, filename, status, timestamps

 **Dedup:** SHA-256 hash of raw bytes is checked in Redis before storing. If hash exists, skip — do not reprocess.

3. End-to-End Pipeline Flow



4. Classification — How It Works

Three layers run in sequence. Each layer is only invoked if the previous one returns low confidence. 80% of documents resolve at L1 or L2.

Layer	Method	Confidence Threshold	Tools Used
L1 — Rules	Match filename, folder path, source URL against regex patterns	> 0.90 → done	Python regex + config.yaml
L2 — Embedding	Embed first 500 tokens, cosine similarity to domain centroids	> 0.75 → done	BGE-large-en-v1.5 + Redis centroids
L3 — LLM	Send text snippet to Gemma-3, return structured JSON label	< 0.75 → use Gemma	Gemma-3 via Ollama (self-hosted)

Output of classification:

- primary_domain: IT | HR | Finance | Marketing | Operations | Unknown
- doc_type: SOP | Policy | Runbook | Ticket | Report | Spreadsheet | Presentation
- sensitivity: public | internal | confidential
- classification_method: rules | embedding | llm (logged for audit)

?

Unknown domain: Documents with L3 confidence < 0.4 are tagged Unknown and added to a human-review queue. They still get processed and indexed — just under an Unknown namespace.

5. File-Type Processing

5a. PDF

PDF Type	Detection	Tool	Output
Digital PDF	pymupdf extracts > 50 chars per page	pymupdf (fitz)	Clean text + tables via pdfplumber/Camelot
Scanned PDF	pymupdf extracts < 50 chars per page	pdf2image → VLM	OCR text, confidence score per word
Mixed PDF	Per-page detection — some digital, some scanned	Both above, per page	Merged text preserving page order

PDF processing steps in order:

- Open with pymupdf. For each page: attempt text extraction.
- If text < 50 chars: convert page to 300 DPI PNG via pdf2image. Run VLM.
- Extract tables: try Camelot (lattice mode) first. If it fails, fall back to pdfplumber stream mode.
- Extract images: pymupdf page.get_images(). Each image stored to MinIO, captioned by Gemma-3 vision.

5b. DOCX (Word)

- Parse with python-docx. Extract paragraphs preserving Heading 1/2/3 hierarchy.
- Extract tables via doc.tables — each table serialized as markdown + JSON.
- Extract embedded images from the internal word/media/ directory (unzip the .docx).

- Images saved to MinIO. Gemma-3 vision generates a one-line caption per image.
- Caption + surrounding paragraph text stored together as a single image chunk.

5c. PPTX (PowerPoint)

- Parse with python-pptx. Process slide by slide.
- Each slide = one chunk. Extract: title, body text, speaker notes.
- For each shape: if it contains an image, extract bytes and save to MinIO.
- Table shapes: extract as markdown table, append to slide chunk.
- Gemma-3 vision captions any slide that has more images than text (image-dominant slides).

5d. XLSX (Excel)

 **Key decision:** Always convert Excel to PDF via LibreOffice first. Then run the PDF pipeline for image extraction. Run openpyxl on the original for structured data.

- LibreOffice headless converts .xlsx to .pdf — captures charts, shapes, floating images faithfully.
- PDF pipeline extracts images from the converted PDF (same as section 5a).
- openpyxl reads each sheet from the original .xlsx — extracts tabular data per sheet.
- Each sheet becomes its own chunk. Column headers are injected into every row-batch chunk for context.
- Empty sheets are skipped. Sheets with < 3 rows are merged into adjacent sheet chunk.

Excel Edge Cases — Fully Handled

Edge Case	How We Handle It
Floating images (charts, diagrams)	LibreOffice → PDF → pymupdf image extraction — 100% captured
Images inside cells (new Excel feature)	LibreOffice renders them → PDF pipeline extracts them
Charts (bar, pie, line)	LibreOffice renders to image in PDF — Gemma-3 vision captions the chart
Merged cells	openpyxl detects merged ranges, unmerges and fills value for each cell
Multiple sheets with same column names	Each sheet chunked independently with sheet name as prefix
Password-protected files	Catch exception, log as PROTECTED, add to manual-review queue
Corrupt .xlsx	Catch exception at openpyxl open(), log as CORRUPT, move to dead-letter
Formula cells	openpyxl reads data_only=True — reads last calculated value, not formula

6. Where Gemma-3 Is Used

Gemma-3 is the primary OCR engine. it does the character recognition. Gemma-3 is used in three specific roles:

Role	Input	What Gemma Does	When Triggered
------	-------	-----------------	----------------

Image captioning	Extracted image (PNG bytes)	One-sentence caption describing the image content for semantic indexing.	Every extracted image
LLM classification	First 500 tokens of document	Return JSON: {domain, doc_type, sensitivity, confidence}	L1 and L2 both below threshold



Deployment: Gemma-3 runs via Ollama on-prem. Model: gemma3:12b for text tasks, gemma3:12b vision variant for image captioning. Single A10 GPU handles all three roles.

7. Chunking Strategy

Deployment: Gemma-3 runs via Ollama on-prem. Model: gemma3:12b for text tasks, gemma3:12b vision variant for image captioning. Single A10 GPU handles all roles.

Document Type	Chunking Method	Chunk Size	Token Limit	Overlap	Justification
Tickets (Jira / ServiceNow)	No splitting — 1 ticket = 1 chunk	Whole doc	≤ 512	None	Tickets are short and self-contained. Splitting breaks context. Must be retrieved as a whole.
SOP / Policy	RecursiveCharacterTextSplitter (tiktoken)	512 tok	512	50 tok	Adhoc prose with no guaranteed structure. Recursive splitting respects natural language flow. tiktoken ensures token-accurate sizing.
Runbooks	AST-aware (code) + Recursive fallback (prose)	200–350 tok	350	0–20 tok	Runbooks mix prose and code. AST splitter preserves function/block boundaries. try/except fallback handles malformed scripts.
Wiki / Confluence	RecursiveCharacterTextSplitter (tiktoken)	512 tok	512	50 tok	Wiki pages vary in length. Same approach as SOPs. Consistent chunk size keeps cross-document retrieval scoring comparable.
Email Threads	RecursiveCharacterTextSplitter (tiktoken)	300 tok	300	40 tok	Short conversational content. Quoted reply chains stripped in preprocessing — only original message text is chunked.
Slack Messages	No splitting — 1 message = 1 chunk	Whole doc	≤ 300	None	Messages are inherently short. Threaded replies optionally concatenated with parent to preserve context.
Tables	Row serialization — 1 row = 1 chunk	Per row	≤ 200	None	Each row serialized as: 'Col1: Val1 Col2: Val2'. Column names embedded in every chunk.
Internal Portals (crawled)	RecursiveCharacterTextSplitter (tiktoken)	512 tok	512	50 tok	Unstructured HTML converted to plain text after stripping nav, footers, boilerplate.

8. Storage After Processing

What	Where	Schema / Path
Raw original file	MinIO	s3://idp/raw/{source}/{doc_id}.{ext}
Extracted text JSON	MinIO	s3://idp/processed/{doc_id}/text.json
Extracted tables	MinIO	s3://idp/processed/{doc_id}/tables.json
Extracted images	MinIO	s3://idp/images/{doc_id}/{index}.png
Document metadata	PostgreSQL	documents table: doc_id, domain, type, status, timestamps
Chunk metadata	PostgreSQL	chunks table: chunk_id, doc_id, chunk_type, keywords, section_title
Image metadata	PostgreSQL	images table: image_id, doc_id, chunk_id, caption, minio_path
Chunk vectors	Qdrant	collection: enterprise_knowledge, dual-vector: content + question
Dedup hashes	Redis	SET idp:hash:{sha256} → doc_id, TTL 30 days
Domain centroids	Redis	KEY idp:centroid:{domain} → numpy array, refreshed nightly

9. Edge Cases — Full Coverage

Scenario	Detection	Action
Scanned PDF	pymupdf text < 50 chars/page	pdf2image → Gemma correction
Corrupt file	Parser throws exception on open	Log as CORRUPT, move to dead-letter queue, alert admin
Password-protected file	Parser throws PermissionError	Log as PROTECTED, skip, add to manual-review queue
Empty file	0 bytes or 0 text extracted	Log as EMPTY, skip indexing, notify source owner
Duplicate document	SHA-256 already in Redis	Skip entirely, update ingested_at timestamp only
Images in Excel	openpyxl cannot extract charts	LibreOffice → PDF → standard image pipeline
Mixed-language document	langdetect returns multiple langs	Split by language segment, tag each chunk with language code
Extremely large file >200MB	File size check before processing	Stream page-by-page, never load full file in memory
Table extraction failure	Camelot raises ParsingError	Fall back to pdfplumber, if both fail treat table area as text
Unknown domain after L3	Gemma confidence < 0.4	Tag as Unknown, index normally, add to human-labeling queue
Image-only PDF (no text)	Zero text pages, all images	Run VLM on every page, treat as fully scanned

10. Failure Handling & Retry

- Every pipeline stage wraps execution in try/except and writes a status record to PostgreSQL pipeline_logs.
- Failed documents are pushed to Kafka topic: idp.dlq (dead-letter queue) with failure_reason tag.
- Retry agent polls idp.dlq every 10 minutes. Max 3 retries per document. Each retry uses a fallback strategy.
- After 3 failures: status set to FAILED, admin alerted via webhook.

Stage Failed	Retry Strategy
Table extraction	Retry with pdfplumber if Camelot failed; retry as plain text if both fail
Classification	Retry with next layer (L1 fail → L2 → L3 → Unknown)
Embedding generation	Retry with batch size halved; if GPU OOM, switch to CPU
Qdrant write	Retry 3x with exponential backoff (2s, 4s, 8s)
Gemma-3 timeout	Skip LLM step, use raw OCR output or L2 classification

Tool Summary

Role	Tool	Why
LLM (text)	Gemma-3 12B via Ollama	Self-hosted, classification + chunk enrichment + OCR correction
LLM (vision)	VIM like Llama maverick and qwen 3.5	Image captioning, chart understanding
Embeddings	BAAI/bge-large-en-v1.5	Best open-source English embeddings, 1536 dims
Reranker	BAAI/bge-reranker-large	Cross-encoder for retrieval precision
Vector DB	Qdrant	Dual-vector, payload filtering, self-hostable
Metadata DB	PostgreSQL	Documents, chunks, images, pipeline logs
Object Store	MinIO	S3-compatible, self-hostable, raw + processed files
Cache / Dedup	Redis	Hash dedup, centroid cache, 30-day TTL
OCR	VIM like Llama maverick and gemma 3, qwen 3.5	Multi-language, table detection, GPU-accelerated
PDF parsing	pymupdf (fitz)	Fast, layout-aware, image extraction
PDF tables	Camelot → pdfplumber (fallback)	Camelot for bordered tables, pdfplumber for borderless
Word parsing	python-docx	Full heading + table + image access
Excel parsing	openpyxl + LibreOffice	openpyxl for data, LibreOffice for chart/image conversion
PPT parsing	python-pptx	Slide-level, shape-level, image extraction
Queue	Apache Kafka	Async processing, replay, dead-letter support
Orchestration	Apache Airflow	DAG pipeline, retry, monitoring

Excel→PDF conversion	LibreOffice headless	Only reliable way to capture charts and floating images
----------------------	----------------------	---